

Artificial Intelligence — Mid-Term Exam Answer Sheet

Nepal College of Information Technology | Mid-Term Spring 2026 | ME/CS

Q1(a) What is Artificial Intelligence? If the Turing Test is passed, does this show that computers exhibit intelligence? State your reasons. (8 marks)

Definition of AI: Artificial Intelligence (AI) is the branch of computer science concerned with building systems (agents) that can perform tasks which normally require human intelligence — such as reasoning, learning, perception, problem-solving, and natural language understanding. Formally, AI can be viewed along two dimensions:

- **Thought vs Behavior:** Systems that think/reason like humans vs systems that act like humans.
- **Human-like vs Rational:** Systems that mimic human performance vs systems that act optimally (rationally) to achieve goals.

This gives the four classical definitions of AI: *Thinking Humanly*, *Thinking Rationally*, *Acting Humanly*, *Acting Rationally* (the last being the modern standard — "rational agents").

The Turing Test: Proposed by Alan Turing (1950) as an operational test for machine intelligence ("The Imitation Game"). A human evaluator converses (via text) with a human and a machine without knowing which is which. If the evaluator cannot reliably distinguish the machine from the human, the machine is said to have passed the test.

Does passing the Turing Test prove intelligence? — Discussion & Reasons

Arguments that it DOES show intelligence:

- It tests functional/behavioral intelligence — if a machine can converse indistinguishably from a human across arbitrary topics (reasoning, humor, general knowledge), it demonstrates a broad, flexible competence that historically only intelligent beings possessed.
- It is an operational, measurable definition rather than a vague philosophical one.

Arguments that it does NOT prove true intelligence (majority view in modern AI):

1. **The Chinese Room Argument (Searle):** A system can manipulate symbols to produce convincing responses without any real understanding or consciousness — it may simulate intelligence without possessing it.
2. **Test measures imitation, not cognition:** Passing the test only shows the machine is good at *appearing* human in conversation, not that it reasons, understands meaning, or has genuine cognitive states.

3. **Narrow scope:** The test only evaluates linguistic/conversational ability, ignoring other aspects of intelligence like perception, physical reasoning, creativity in the real world, and learning from experience.
4. **Gaming the test:** Chatbots can pass by using tricks (evasion, humor, feigned typos) rather than real reasoning — e.g., "Eugene Goostman" chatbot's controversial "pass" in 2014.
5. Modern Large Language Models (LLMs) can produce human-like, fluent conversation through statistical pattern learning from massive data, without any grounded understanding of the real world — showing that fluent behavior can be decoupled from genuine comprehension.

Conclusion: Passing the Turing Test demonstrates that a machine can convincingly *simulate* intelligent behavior, but it does not conclusively prove the machine truly *understands, thinks,* or is *conscious* in the way humans are. It is a useful benchmark for behavioral/functional AI but not a sufficient philosophical proof of intelligence.

Q1(b) For the following graph, compute A* search. Assume start state is S and goal state is G. (7 marks)

Given Graph (edges and path costs):

S → A : 1	A → B : 3	B → D : 3
S → G : 12	A → C : 1	C → D : 1
		C → G : 2
		D → G : 3

Heuristic values (h_i):

State	S	A	B	C	D	G
h_i	5	3	6	2	3	0

A* Search Formula: $f(n) = g(n) + h(n)$

- $g(n)$ = actual cost from start S to node n
- $h(n)$ = estimated (heuristic) cost from n to goal G

Step-by-step Execution:

Step 1: Expand S (start node).

- $g(S)=0, h(S)=5, f(S)=5$

- Frontier after expanding S:
 - A: $g=0+1=1$, $h=3$, **$f=4$**
 - G: $g=0+12=12$, $h=0$, **$f=12$**

Step 2: Choose lowest $f \rightarrow A$ (**$f=4$**). Expand A.

- Frontier: B, C (from A), and G (from S) remain
 - B: $g=1+3=4$, $h=6$, **$f=10$**
 - C: $g=1+1=2$, $h=2$, **$f=4$**
 - G (via S): $f=12$ (*still in frontier*)

Step 3: Choose lowest $f \rightarrow C$ (**$f=4$**). Expand C.

- Frontier updated:
 - D: $g=2+1=3$, $h=3$, **$f=6$**
 - G (via C): $g=2+2=4$, $h=0$, **$f=4$**
 - B: $f=10$, G (via S): $f=12$ (*dominated — higher cost*)

Step 4: Choose lowest $f \rightarrow G$ (**$f=4$** , via C). G is the goal node $\rightarrow$ **Stop**.

Search Tree Summary Table:

Node Expanded	$g(n)$	$h(n)$	$f(n) = g+h$
S	0	5	5
A	1	3	4
C	2	2	4
G	4	0	4

Optimal Path Found: $S \rightarrow A \rightarrow C \rightarrow G$

Total Path Cost: $1 + 1 + 2 = 4$

(Verification — other possible paths: $S \rightarrow G = 12$; $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = 1+3+3+3 = 10$; $S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 1+1+1+3 = 6$. The path $S \rightarrow A \rightarrow C \rightarrow G$ with cost 4 is indeed the minimum, confirming A^* found the optimal solution.)

Q2(a) Water Jug Problem — get exactly 2 gallons into the 4-gallon jug using a production rule system. (7 marks)

Problem Formulation:

- State representation: (x, y) where x = amount of water in the 4-gallon jug (0-4), y = amount in the 3-gallon jug (0-3).
- **Initial State:** $(0, 0)$
- **Goal State:** $(2, y)$ for any y — i.e., 4-gallon jug contains exactly 2 gallons.

Production Rules (Operators):

Rule	Condition	Action	Resulting State
R1	$x < 4$	Fill 4-gallon jug	$(4, y)$
R2	$y < 3$	Fill 3-gallon jug	$(x, 3)$
R3	$x > 0$	Empty 4-gallon jug	$(0, y)$
R4	$y > 0$	Empty 3-gallon jug	$(x, 0)$
R5	$x+y \geq 4, y > 0$	Pour 3-gal into 4-gal until 4-gal full	$(4, y-(4-x))$
R6	$x+y \geq 3, x > 0$	Pour 4-gal into 3-gal until 3-gal full	$(x-(3-y), 3)$
R7	$x+y \leq 4, y > 0$	Pour all of 3-gal into 4-gal	$(x+y, 0)$
R8	$x+y \leq 3, x > 0$	Pour all of 4-gal into 3-gal	$(0, x+y)$

Solution Trace (applying production rules):

Step	State (x,y)	Rule Applied
0	(0, 0)	Initial state
1	(4, 0)	R1: Fill 4-gallon jug
2	(1, 3)	R6: Pour 4-gal → 3-gal until 3-gal full
3	(1, 0)	R4: Empty 3-gallon jug
4	(0, 1)	R8: Pour all of 4-gal into 3-gal
5	(4, 1)	R1: Fill 4-gallon jug
6	(2, 3)	R6: Pour 4-gal → 3-gal until 3-gal full

Goal Reached: At state (2, 3), the 4-gallon jug contains **exactly 2 gallons** ✓

Explanation of final step: In state (4,1), the 3-gallon jug needs 2 more gallons to become full ($3-1=2$). Pouring from the 4-gallon jug (which has 4) into the 3-gallon jug transfers 2 gallons, leaving $4-2 = 2$ gallons in the 4-gallon jug, and the 3-gallon jug becomes full (3).

Q2(b) Write an algorithm for Backtracking Search for CSPs. Use backtracking to solve: BASE + BALL = GAMES (8 marks)

Backtracking Search Algorithm for CSP:

```

function BACKTRACKING-SEARCH(csp):
    return BACKTRACK({}, csp)

function BACKTRACK(assignment, csp):
    if assignment is complete:
        return assignment
    var = SELECT-UNASSIGNED-VARIABLE(csp, assignment)
    for value in ORDER-DOMAIN-VALUES(var, assignment, csp):
        if value is consistent with assignment (satisfies constraints):
            add {var = value} to assignment
            inferences = INFERENCE(csp, var, value)    // e.g. forward checking
            if inferences ≠ failure:
                add inferences to assignment
                result = BACKTRACK(assignment, csp)
                if result ≠ failure:
                    return result
            remove {var = value} and inferences from assignment
    return failure

```

Solving the Cryptarithmic Problem:

```

  B A S E
+ B A L L
-----
G A M E S

```

Step 1 — Formulate as a CSP:

- **Variables:** B, A, S, E, L, G, M (7 unique letters)
- **Domain:** {0, 1, 2, ..., 9} for each variable
- **Constraints:**
 1. All-Diff: B, A, S, E, L, G, M must all take **different** digits.
 2. Leading digits cannot be 0: $B \neq 0$, $G \neq 0$.
 3. Arithmetic constraint (column-wise addition with carries c_1, c_2, c_3, c_4):
 - Units: $E + L = S + 10 \cdot c_1$
 - Tens: $S + L + c_1 = E + 10 \cdot c_2$
 - Hundreds: $A + A + c_2 = M + 10 \cdot c_3$
 - Thousands: $B + B + c_3 = G + 10 \cdot c_4$

- Ten-thousands: $c_4 = G$

Step 2 — Backtracking Search Process (reasoning):

- Since GAMES has 5 digits while BASE and BALL have only 4 digits each, the addition **must produce a carry-out**, so $c_4 = G = 1$ (the only possible carry from adding two digits + carry is 0 or 1).
- Assign $G = 1$. Backtrack-check: satisfies $G \neq 0$. ✓ Continue.
- From Thousands column: $2B + c_3 = A + 10$. Try values for B (excluding 0,1): backtracking tries $B=2,3,4,5\dots$ checking consistency with the Hundreds and Units/Tens columns at each step, rejecting assignments that cause constraint violations (e.g., repeated digits) and backtracking whenever a dead-end is reached.
- Continuing systematic assignment + constraint-checking (forward checking eliminates conflicting domain values early), the search converges on a consistent, all-different assignment satisfying every column constraint.

Step 3 — Verified Solution:

Letter	B	A	S	E	L	G	M
Digit	7	4	8	3	5	1	9

Verification:

```

  7 4 8 3   (BASE)
+ 7 4 5 5   (BALL)
-----
 1 4 9 3 8   (GAMES)

```

- Units: $3+5 = 8 = S$ ✓ (carry 0)
- Tens: $8+5+0 = 13 \rightarrow$ write 3 (=E), carry 1 ✓
- Hundreds: $4+4+1 = 9 = M$ ✓ (carry 0)
- Thousands: $7+7+0 = 14 \rightarrow$ write 4 (=A), carry 1 ✓
- Ten-thousands: carry 1 = G ✓

$7483 + 7455 = 14938$ ✓ — All digits distinct, all constraints satisfied.

Q3(a) Compare uninformed and informed search techniques in terms of space and time complexity. (8 marks)

Basis	Uninformed (Blind) Search	Informed (Heuristic) Search
Definition	Explores the search space without any domain-specific knowledge about how close a state is to the goal	Uses a heuristic function $h(n)$ to estimate distance/cost to goal, guiding the search intelligently
Examples	BFS, DFS, Uniform Cost Search (UCS), Depth-Limited Search, Iterative Deepening	A* Search, Greedy Best-First Search, Hill Climbing
Information Used	Only problem definition (initial state, actions, goal test)	Problem definition + heuristic function $h(n)$
Time Complexity	Generally higher — e.g., BFS: $O(b^d)$, DFS: $O(b^m)$ (b =branching factor, d =depth, m =max depth)	Generally lower due to guided search — A*: $O(b^d)$ in worst case, but in practice explores far fewer nodes; Greedy: can be $O(b^m)$ worst case but often much faster
Space Complexity	Often high — BFS: $O(b^d)$ (stores all nodes at a level); UCS similar	A* also needs $O(b^d)$ in worst case (stores frontier + explored nodes), but explores fewer nodes than uninformed search practically
Optimality	BFS/UCS are optimal (UCS with non-negative costs); DFS is not optimal	A* is optimal if heuristic is admissible (never overestimates); Greedy Best-First is not guaranteed optimal
Completeness	BFS, UCS are complete; DFS incomplete in infinite spaces	A* is complete if heuristic is admissible and search space finite
Efficiency	Inefficient for large search spaces — "blind" exploration wastes effort on irrelevant directions	Much more efficient — heuristic prunes irrelevant branches, focusing search toward the goal
Use Case	Small search spaces, or when no domain knowledge/heuristic is available	Large search spaces where a good heuristic can be defined (e.g., pathfinding, puzzle solving)

Summary: Uninformed search guarantees finding a solution (if one exists) but can be computationally expensive since it explores blindly. Informed search leverages heuristic

knowledge to reduce the number of nodes explored, significantly improving time and often space efficiency — provided a good heuristic function is available for the problem domain.

Q3(b) Explain Deep Learning and what are their applications. (7 marks)

Definition: Deep Learning is a subfield of Machine Learning based on **artificial neural networks with multiple (deep) layers** that automatically learn hierarchical feature representations directly from raw data, without requiring manual feature engineering.

Key Characteristics:

- Uses architectures like **Multi-Layer Perceptrons (MLP), CNNs, RNNs/LSTMs, and Transformers**.
- Learns features layer by layer: early layers learn simple/low-level features, deeper layers combine these into abstract, high-level concepts.
- Requires **large amounts of labeled data** and significant computational power (GPUs/TPUs) for training via backpropagation and gradient descent.
- "Deep" refers to the number of hidden layers stacked between input and output.

Why Deep Learning over traditional Machine Learning:

- Traditional ML relies on hand-crafted features (feature engineering) designed by domain experts.
- Deep Learning automatically discovers relevant features from raw data (e.g., raw pixels, raw audio, raw text), which scales much better with large, complex datasets.

Applications of Deep Learning:

1. **Computer Vision:** Image classification, object detection, facial recognition, medical image diagnosis (e.g., detecting tumors in X-rays/MRI using CNNs).
2. **Natural Language Processing:** Machine translation, sentiment analysis, chatbots, text summarization (using RNN/LSTM/Transformer models like BERT, GPT).
3. **Speech Recognition:** Voice assistants (Siri, Alexa, Google Assistant), automatic transcription.
4. **Autonomous Vehicles:** Self-driving cars use deep learning for lane detection, obstacle recognition, and decision-making.
5. **Recommendation Systems:** Netflix, YouTube, and Amazon use deep learning to personalize content/product recommendations.
6. **Healthcare:** Disease prediction, drug discovery, personalized treatment recommendation.
7. **Fraud Detection:** Banks and fintech companies use deep learning to detect anomalous/fraudulent transactions.

8. **Generative AI:** Image generation (GANs, diffusion models), text generation (LLMs like GPT), used in art, content creation, and code generation.

Deep Learning's ability to automatically learn complex, hierarchical patterns from massive datasets has made it the driving force behind most modern breakthroughs in AI.

Q4(a) Ecommerce Chatbot Scenario (7+8 marks)

Scenario: Hired by an ecommerce company of Nepal to develop a chatbot to answer customer queries posted through its website.

i. What do you think will be the issues with text complaints posted?

Handling free-text customer complaints poses several NLP challenges:

1. **Language Ambiguity:** Customer messages may be lexically or syntactically ambiguous (e.g., "order not received" could mean delayed, lost, or wrong item).
2. **Spelling & Grammar Errors:** Real customer text is often informal, with typos, abbreviations, and poor grammar (e.g., "my ordr nt cme yet").
3. **Code-Mixing / Multilingual Text:** In Nepal's context, customers may type in **Romanized Nepali**, mixed Nepali-English ("mero order aayena"), which standard English NLP models struggle with.
4. **Sarcasm & Emotion Detection:** Detecting frustration, urgency, or sarcasm in complaints is difficult but important for prioritizing responses.
5. **Ambiguous Intent:** A single complaint may contain multiple issues (e.g., wrong item AND late delivery) requiring multi-intent classification.
6. **Out-of-Vocabulary/Domain-specific Terms:** Product names, brand-specific terms not in general language models.
7. **Context/History Dependence:** Understanding a complaint may require previous order history/context, not just the text itself.

ii. Discuss about the steps of NLP to solve this problem.

1. **Text Preprocessing:** Tokenization, lowercasing, removing stop-words, handling emojis/special characters, spelling correction.
2. **Language Detection / Normalization:** Detect if the text is in English, Nepali, or Romanized Nepali, and normalize accordingly.
3. **Lexical & Syntactic Analysis:** Tokenize and POS-tag the complaint to understand grammatical structure.
4. **Named Entity Recognition (NER):** Extract key entities — order ID, product name, date, delivery location.

5. **Intent Classification:** Use a trained classifier (e.g., fine-tuned Transformer/BERT model) to categorize the complaint (e.g., "late delivery", "wrong item", "refund request", "damaged product").
6. **Sentiment Analysis:** Determine the emotional tone/urgency of the complaint to prioritize response (angry/urgent complaints escalated faster).
7. **Semantic Analysis / Slot Filling:** Extract structured information needed to resolve the complaint (order number, issue type, desired resolution).
8. **Response Generation:** Generate an appropriate automated reply (template-based or generative) or route the complaint to the correct human support team.
9. **Continuous Learning/Feedback Loop:** Retrain the model periodically using new labeled complaint data to improve accuracy over time.

iii. Can we use LLMs in this scenario?

Yes. Large Language Models (LLMs like GPT, Claude, or fine-tuned BERT variants) are highly suitable for this scenario:

Advantages of using LLMs:

- **Robust language understanding:** LLMs handle spelling errors, informal text, and even code-mixed Nepali-English better than traditional rule-based NLP systems.
- **Multi-intent & context understanding:** Can understand complex complaints with multiple issues and maintain conversational context across multiple turns.
- **Few-shot/Zero-shot capability:** LLMs can classify and respond to new types of complaints without needing large amounts of labeled training data.
- **Natural response generation:** Can generate empathetic, human-like, and contextually appropriate replies rather than rigid templates.
- **Multilingual support:** Modern LLMs can be prompted/fine-tuned to understand and respond in Nepali as well as English.

Considerations/Limitations:

- **Cost:** API calls to large LLMs can be expensive at scale for high query volumes.
- **Hallucination risk:** LLMs may generate plausible-sounding but incorrect information (e.g., wrong refund policy) — must be grounded with **Retrieval-Augmented Generation (RAG)** using the company's actual policies/order database.
- **Latency:** Larger models may respond slower than lightweight, specialized classifiers for simple/high-volume queries.
- **Data privacy:** Customer order data must be handled securely/compliant with data protection norms.

Recommended Approach: A hybrid system — use an LLM (with RAG grounded in the company's order database and policies) for understanding and generating responses, combined with a lightweight intent classifier for fast triage of simple/common queries, and human escalation for complex or sensitive complaints.

Q4(b) Write short notes on:

i. Learning

Learning, in AI, refers to the ability of a system to improve its performance on a task through experience, without being explicitly reprogrammed for every scenario. It is the core process behind Machine Learning.

Types of Learning:

1. **Supervised Learning:** The model learns from **labeled data** (input-output pairs), mapping inputs to known correct outputs. Example: Email spam classification (emails labeled "spam"/"not spam").
2. **Unsupervised Learning:** The model finds hidden patterns/structures in **unlabeled data** (no predefined output). Example: Customer segmentation via clustering (K-Means).
3. **Reinforcement Learning:** An agent learns by interacting with an environment, receiving **rewards or penalties** for actions, and improving its policy to maximize cumulative reward over time. Example: A game-playing agent (AlphaGo) learning through trial and error.
4. **Semi-Supervised Learning:** Uses a small amount of labeled data combined with a large amount of unlabeled data.

Why Learning Matters in AI: It allows AI systems to generalize to new, unseen situations, adapt to changing environments, and improve automatically over time — the foundation of modern AI capabilities like image recognition, NLP, and recommendation systems.

ii. Knowledge and its Representation

Knowledge in AI refers to facts, information, rules, and relationships about the world that an intelligent agent uses to reason and make decisions.

Knowledge Representation (KR) is the field of AI concerned with how to encode this knowledge in a form that a computer system can process, reason with, and use to solve problems.

Key Requirements of a good KR scheme:

- **Representational Adequacy:** Ability to represent all necessary knowledge.
- **Inferential Adequacy:** Ability to derive new knowledge from existing knowledge.

- **Inferential Efficiency:** Ability to guide reasoning in efficient directions.
- **Acquisitional Efficiency:** Ease of acquiring new knowledge.

Common Knowledge Representation Techniques:

1. **Logical Representation:** Using Propositional Logic or First-Order Logic (FOL) to represent facts as formal logical statements and use inference rules (e.g., resolution) to derive conclusions.
2. **Semantic Networks:** Graph-based representation where nodes represent concepts/objects and edges represent relationships (e.g., "is-a", "has-a").
3. **Frames:** Structured representation (like a record/object) that groups related knowledge about an object/concept as attribute-value pairs (slots).
4. **Production Rules:** IF-THEN rules representing knowledge as condition-action pairs (used in expert systems).
5. **Ontologies:** Formal, structured representation of concepts, categories, and relationships within a domain, enabling shared understanding and reasoning across systems.

Example: Representing "A canary is a bird, and birds can fly" —

- Logic: $\text{Bird}(\text{Canary}), \forall x \text{ Bird}(x) \rightarrow \text{CanFly}(x)$
- Semantic Network: $\text{Canary} \text{ --is_a--> Bird --can--> Fly}$

Effective knowledge representation is essential because it directly determines how well an AI system can reason, answer queries, and solve real-world problems.

End of Answer Sheet. Good luck with your mid-term!